# IT Helpdesk & Asset Management System

Osobny plan projektu pod portfolio, GitHub i CV

## 1. Główna idea projektu

Webowy system dla wewnętrznego działu IT w firmie. Pracownik tworzy zgłoszenie, technik IT przyjmuje je do obsługi, zmienia status, komentuje, łączy je ze sprzętem użytkownika i zamyka sprawę. Admin zarządza użytkownikami, sprzętem, kategoriami i obserwuje raporty na dashboardzie.

Projekt ma wyglądać jak realny system firmowy, a nie akademicki CRUD. Najważniejsze elementy to proces obsługi zgłoszeń, role użytkowników, historia zmian, SLA, relacyjna baza danych, API, testy i wygodny sposób uruchomienia.

### 1.1. Proponowana nazwa repo

```
it-helpdesk-asset-management-system
```

### 1.2. Stack technologiczny

| Warstwa | Technologia | Dlaczego warto |
|---|---|---|
| Backend | ASP.NET Core Web API | Pokazuje C#/.NET w webowym, firmowym kontekście. |
| Frontend | React + TypeScript | Czytelny panel administracyjny i nowoczesny frontend. |
| Baza danych | PostgreSQL | Relacyjny model danych, zapytania, migracje i raporty. |
| ORM | Entity Framework Core | Migracje, relacje, walidacje i praca z bazą w .NET. |
| Autoryzacja | JWT + role-based authorization | Realistyczne role: Employee, Technician, Admin. |
| Testy backendu | xUnit + WebApplicationFactory + Testcontainers | Unit i integration tests z prawdziwym PostgreSQL w kontenerze. |
| Testy E2E | Playwright + TypeScript | Scenariusze użytkownika przez frontend. |
| DevOps | Docker Compose + GitHub Actions | Jedna komenda do uruchomienia i automatyczny build/test w CI. |

### 1.3. Cel rekrutacyjny

- Pokazać, że potrafisz zbudować system bliski realnym procesom w firmie.
- Połączyć aplikację webową, API, bazę danych, role, testy, dokumentację i lokalny deployment.
- Wzmocnić aplikacje na stanowiska: IT Support Intern, Application Support, Technical Support Engineer, QA Intern, Junior QA, Junior .NET Developer Intern, SQL/Database Support, Implementation Consultant Trainee.
- Dać rekruterowi szybki do zrozumienia przykład: system do zgłoszeń IT i zarządzania sprzętem.

## 2. Zakres funkcjonalny

### 2.1. Użytkownicy i role

| Rola | Uprawnienia |
|---|---|
| Employee | Tworzy zgłoszenia, widzi swoje zgłoszenia, dodaje publiczne komentarze, potwierdza rozwiązanie lub zamyka zgłoszenie. |
| Technician | Widzi kolejkę zgłoszeń, przypisuje zgłoszenia do siebie, zmienia status, priorytet, dodaje komentarze publiczne i wewnętrzne, wiąże zgłoszenia ze sprzętem. |
| Admin / Manager | Zarządza użytkownikami, rolami, sprzętem, kategoriami, widzi dashboard, raporty i pełny audyt. |

Dla portfolio najlepiej przygotować seedowane konta testowe, aby osoba sprawdzająca projekt mogła od razu zobaczyć różnice między rolami.

```
employee@test.com / Password123!
technician@test.com / Password123!
admin@test.com / Password123!
```

## 2.2. Moduł zgłoszeń IT

Ticket jest sercem systemu. Powinien mieć pola pozwalające przejść pełny proces obsługi problemu technicznego.

```
Ticket
- Id
- Title
- Description
- Category
- Priority
- Status
- CreatedAt
- UpdatedAt
- ResolvedAt
- CreatedByUserId
- AssignedTechnicianId
- RelatedAssetId
- SlaDueAt
- IsOverdue
```

| Typ | Wartości |
|---|---|
| Kategorie | Hardware, Software, Network, Account/Access, Printer, Email, Other |
| Priorytety | Low, Medium, High, Critical |
| Statusy | New, Assigned, InProgress, WaitingForUser, Resolved, Closed, Cancelled |

## 2.3. Workflow zgłoszenia

```
Employee tworzy zgłoszenie
-> Ticket dostaje status New
-> Technician przypisuje zgłoszenie do siebie
-> Status zmienia się na Assigned
-> Technician zaczyna pracę
-> Status InProgress
-> Technician dodaje komentarz
-> Status Resolved
-> Employee albo Admin zamyka zgłoszenie
-> Status Closed
```

Ten workflow sprawia, że aplikacja nie jest tylko zbiorem formularzy. Pokazuje logikę biznesową i kontrolę procesu.

## 2.4. Komentarze do zgłoszeń

```
TicketComment
- Id
- TicketId
- AuthorId
- Body
- CreatedAt
- IsInternal
```

- Komentarz publiczny: widoczny dla pracownika, technika i admina.
- Komentarz wewnętrzny: widoczny tylko dla technika i admina.
- Komentarze powinny pojawiać się chronologicznie w szczegółach zgłoszenia.
- Dodanie komentarza powinno tworzyć wpis w historii zgłoszenia albo audit logu.

## 2.5. Historia zmian i audit log

To jeden z najmocniejszych elementów projektu. W prawdziwych systemach supportowych ważne jest, kto zmienił status, priorytet, przypisanie albo powiązany sprzęt.

```
AuditLog
- Id
- EntityType
- EntityId
- Action
- OldValue
- NewValue
- UserId
- CreatedAt
```

- Ticket created
- Status changed from New to Assigned
- Priority changed from Medium to High
- Assigned technician changed
- Asset linked to ticket
- Comment added
- Ticket resolved
- Ticket closed

## 2.6. SLA

SLA może być proste, ale powinno być widoczne w systemie i przetestowane. Przy tworzeniu zgłoszenia system oblicza SlaDueAt na podstawie priorytetu.

| Priority | Czas na rozwiązanie |
|---|---|
| Low | 72h |
| Medium | 48h |
| High | 24h |
| Critical | 4h |

```
SlaDueAt = CreatedAt + czas_zależny_od_priorytetu
IsOverdue = Status != Resolved/Closed && currentTime > SlaDueAt
```

Na liście zgłoszeń można pokazać: Due in 3h 20m albo Overdue by 1h 15m.

## 2.7. Asset Management

Drugi duży moduł to zarządzanie sprzętem IT. Dzięki temu projekt trafia nie tylko w development, ale też w IT support i procesy firmowe.

```
Asset
- Id
```

- AssetTag
- Type
- Brand
- Model
- SerialNumber
- Status
- AssignedToUserId
- PurchaseDate
- WarrantyUntil
- Notes

| Element | Wartości / funkcje |
|---|---|
| Typy sprzętu | Laptop, Desktop, Monitor, Phone, Printer, Router, DockingStation, Other |
| Statusy sprzętu | Available, Assigned, InRepair, Retired, Lost |
| Funkcje | Dodawanie sprzętu, przypisanie do użytkownika, zmiana statusu, powiązanie sprzętu ze zgłoszeniem. |

Przykład realnego scenariusza: użytkownik ma przypisany laptop, zgłasza przegrzewanie, technik wiąże ticket z laptopem, oznacza sprzęt jako InRepair, rozwiązuje zgłoszenie i przywraca status Assigned.

## 2.8. Dashboard i raporty

| Widok | Metryki |
|---|---|
| Technician | My open tickets, Unassigned tickets, Overdue tickets, Tickets by priority |
| Admin | Total open tickets, Resolved this week, Average resolution time, Tickets by category, Assets by status, Top users by number of tickets |

Open tickets: 18
Overdue tickets: 3
Resolved this week: 12
Average resolution time: 9h 45m
Available assets: 24
Assets in repair: 4

## 2.9. Minimalny zakres MVP

[ ] Login na seedowanych kontach
[ ] Role: Employee, Technician, Admin
[ ] Tworzenie zgłoszeń
[ ] Lista zgłoszeń z filtrowaniem po statusie, priorytecie, kategorii i przypisaniu
[ ] Widok szczegółów zgłoszenia
[ ] Komentarze publiczne i wewnętrzne
[ ] Zmiana statusu i przypisanie technika
[ ] Prosta logika SLA
[ ] Lista sprzętu
[ ] Przypisanie sprzętu do użytkownika
[ ] Powiązanie sprzętu ze zgłoszeniem
[ ] Historia zmian zgłoszenia
[ ] Dashboard z podstawowymi statystykami
[ ] Testy backendu i kilka testów E2E
[ ] Docker Compose
[ ] README ze screenami, opisem i instrukcją uruchomienia

## 2.10. Funkcje na później

- Powiadomienia mailowe po zmianie statusu

- Załączniki do zgłoszeń
- Eksport raportów do CSV/PDF
- Tagi zgłoszeń i bardziej rozbudowane filtrowanie
- Import sprzętu z CSV
- SignalR/live updates dla kolejki zgłoszeń
- Integracja z LDAP/Active Directory jako opis koncepcyjny, niekoniecznie implementacja

# 3. Model danych i endpointy API

## 3.1. Proponowane tabele

```
Users
Roles
Tickets
TicketComments
TicketHistory albo AuditLogs
Categories
Assets
AssetAssignments
```

## 3.2. Relacje

```
User 1 -> many Tickets created by user
User 1 -> many Tickets assigned to technician
Ticket 1 -> many TicketComments
Ticket 1 -> many TicketHistory entries
User 1 -> many Assets assigned to user
Asset 1 -> many Tickets related to asset
```

## 3.3. Endpointy API

| Moduł | Endpointy |
|---|---|
| Auth | POST /api/auth/login<br>GET /api/auth/me |
| Tickets | GET /api/tickets<br>GET /api/tickets/{id}<br>POST /api/tickets<br>PATCH /api/tickets/{id}/status<br>PATCH /api/tickets/{id}/assign<br>PATCH /api/tickets/{id}/priority<br>POST /api/tickets/{id}/comments<br>GET /api/tickets/{id}/history |
| Assets | GET /api/assets<br>GET /api/assets/{id}<br>POST /api/assets<br>PUT /api/assets/{id}<br>PATCH /api/assets/{id}/assign<br>PATCH /api/assets/{id}/status |
| Dashboard | GET /api/dashboard/summary<br>GET /api/dashboard/tickets-by-category<br>GET /api/dashboard/assets-by-status |
| Admin | GET /api/users<br>POST /api/users<br>PATCH /api/users/{id}/role |

# 4. Widoki frontendowe

Frontend ma wyglądać jak prosty, techniczny panel firmowy. W tym projekcie nie chodzi o artystyczną stronę, tylko o czytelny interface do pracy.

| Ścieżka | Cel widoku |
|---|---|
| /login | Logowanie na seedowane konta testowe. |
| /dashboard | Karty z metrykami i podstawowe wykresy/tabele. |
| /tickets | Lista zgłoszeń, filtrowanie, sortowanie, wyszukiwanie. |
| /tickets/new | Formularz tworzenia zgłoszenia. |
| /tickets/:id | Szczegóły zgłoszenia, komentarze, historia, akcje. |
| /assets | Lista sprzętu IT. |
| /assets/:id | Szczegóły sprzętu, przypisanie, historia powiązań. |
| /users | Zarządzanie użytkownikami dla admina. |
| /settings/categories | Kategorie zgłoszeń i konfiguracja podstawowa. |

### 4.1. Lista zgłoszeń

Kolumny:
ID | Title | Category | Priority | Status | Assigned To | Created By | SLA | Created At

Filtry:
Status | Priority | Category | Assigned technician | Only overdue | Search by title

### 4.2. Szczegóły zgłoszenia

Lewy panel:
Title, Description, Status, Priority, Category, Created by, Assigned technician, Related asset, SLA due date

Dolny/prawy panel:
Comments, History, Actions

Akcje:
Assign to me, Change status, Change priority, Link asset, Add comment, Resolve ticket, Close ticket

## 5. Testy Helpdesku

Testy powinny być jednym z głównych atutów projektu. Nie chodzi o ogromną liczbę testów, tylko o sensowne pokrycie logiki biznesowej, API i najważniejszych scenariuszy użytkownika.

### 5.1. Unit tests

[ ] SLA calculation for Low priority returns CreatedAt + 72h
[ ] SLA calculation for Critical priority returns CreatedAt + 4h
[ ] Ticket can move from New to Assigned
[ ] Ticket can move from Assigned to InProgress
[ ] Ticket cannot move from Closed to InProgress
[ ] Only technician can assign ticket
[ ] Asset cannot be assigned to two users at the same time
[ ] ResolvedAt is set when status changes to Resolved
[ ] Closed ticket cannot be modified by Employee
[ ] Overdue flag is false for Resolved and Closed tickets

### 5.2. Integration tests

Testują API, autoryzację i bazę danych. Najlepiej użyć xUnit, WebApplicationFactory i Testcontainers for PostgreSQL.

[ ] POST /api/tickets creates a ticket in database
[ ] GET /api/tickets returns only user's tickets for Employee
[ ] GET /api/tickets returns queue for Technician
[ ] Technician can assign ticket to himself
[ ] Employee cannot assign ticket
[ ] Admin can create asset
[ ] AssetTag must be unique
[ ] Ticket history is created after status change
[ ] Comment is saved and returned in ticket details
[ ] Internal comments are hidden from Employee
[ ] Dashboard returns correct number of open tickets
[ ] JWT-protected endpoint returns 401 without token

### 5.3. E2E tests

Testy przez przeglądarkę. Wystarczy 5-8 dobrze opisanych scenariuszy w Playwright.

[ ] Employee logs in and creates a new ticket
[ ] Technician logs in, sees unassigned ticket and assigns it
[ ] Technician changes ticket status to InProgress
[ ] Technician adds a public comment
[ ] Technician adds an internal comment and Employee cannot see it
[ ] Technician resolves ticket
[ ] Admin logs in and creates a new asset
[ ] Admin assigns asset to employee
[ ] Technician links asset to ticket
[ ] Dashboard shows updated open/resolved ticket count

## 6. Docker, CI/CD, README i Definition of Done

### 6.1. Docker Compose

Projekt powinien dać się uruchomić lokalnie możliwie prosto. Minimalny wariant: frontend, backend i PostgreSQL.

```
docker compose up --build

Services:
- frontend
- backend
- postgres
```

### 6.2. GitHub Actions

- Build backendu
- Uruchomienie unit tests
- Uruchomienie integration tests
- Build frontendu
- Opcjonalnie: E2E tests w osobnym jobie

### 6.3. Struktura README

[ ] Krótki opis projektu i celu biznesowego
[ ] Stack technologiczny
[ ] Lista funkcjonalności

[ ] Screeny głównych ekranów
[ ] Seedowane konta testowe
[ ] Instrukcja uruchomienia z Docker Compose
[ ] Opis architektury i diagram bazy danych
[ ] Opis testów i komenda do uruchomienia
[ ] Sekcja: What I was responsible for
[ ] Sekcja: Possible future improvements

### 6.4. Definition of Done

[ ] Aplikację da się odpalić jedną komendą z README
[ ] Istnieją seedowane konta dla Employee, Technician i Admin
[ ] Można przejść cały proces od stworzenia zgłoszenia do zamknięcia
[ ] Można dodać i przypisać sprzęt
[ ] Można powiązać sprzęt ze zgłoszeniem
[ ] Dashboard pokazuje realne dane z bazy
[ ] Historia zmian działa dla statusów, priorytetów i przypisań
[ ] Jest minimum 10-15 testów backendowych
[ ] Jest minimum 5 testów E2E
[ ] GitHub Actions wykonuje build i testy
[ ] README zawiera screeny, opis architektury i instrukcję uruchomienia

## 7. Opis do CV/GitHuba

IT Helpdesk & Asset Management System
ASP.NET Core, React, TypeScript, PostgreSQL, Entity Framework Core, Docker

Web application for managing internal IT support tickets and company assets. Includes role-based access control, ticket workflow, comments, SLA calculation, audit history, asset assignment, dashboard statistics, PostgreSQL relational database, API tests, E2E tests and Docker-based local setup.

### 7.1. Proponowana kolejność pracy

| Etap | Zakres |
|---|---|
| Tydzień 1 | Backend start, modele, PostgreSQL, migracje, seedowane konta, auth i role. |
| Tydzień 2 | Tickets, komentarze, statusy, historia zmian, SLA, testy unit/integration. |
| Tydzień 3 | Asset management, przypisania sprzętu, dashboard, filtrowanie. |
| Tydzień 4 | Frontend polish, E2E, Docker Compose, GitHub Actions, README, screeny, opis do portfolio. |

### 7.2. Najważniejsza zasada

Nie chodzi o liczbę funkcji. Chodzi o to, żeby projekt dawał rekruterowi jasny dowód: umiesz zbudować, uruchomić, przetestować, opisać i utrzymywać aplikację z realnym procesem biznesowym.